

Interview - реверс crackme

Crackme.exe

Нужно найти для своего email правильный серийный номер.
В качестве результатов нужно прислать email+серийный номер и небольшой отчет со скриншотами, где будет видно ход того, как реверсил, какие алгоритмы смог распознать, как находил серийный номер.

Валидная пара

```
email: temp@example.com0000
Serial: 34EF4F43222FE6BB8A362EDB79383BE2
```

Процесс решения

property	value	value	value	value	value	value
section	section[0]	section[1]	section[2]	section[3]	section[4]	section[5]
name	.text	.rdata	.data	.tls	.rsrc	.reloc
section > sha256	A3CCB9DDC4E7672398E755...	7DDBE5CA0273F310A14AFD...	1C5B8871AFE5A79A6B1FC...	4C6474903705CB450BB6434...	8BA0B978E4DC55DEF047F85...	A74B43FA155739D3CD56347...
entropy	6.210	5.003	2.686	0.020	6.049	6.003
file > ratio (97.70%)	18.39 %	18.39 %	1.15 %	1.15 %	54.02 %	4.60 %
raw-address (begin)	0x00004000	0x00002400	0x00004400	0x00004600	0x00004800	0x0000A600
raw-address (end)	0x00002400	0x00004400	0x00004600	0x00004800	0x0000A600	0x0000AE00
raw-size (43520 bytes)	0x00002000 (8192 bytes)	0x00002000 (8192 bytes)	0x00002000 (512 bytes)	0x00002000 (512 bytes)	0x00005E00 (24064 bytes)	0x00000800 (2048 bytes)
virtual-address (begin)	0x00001000	0x00003000	0x00005000	0x00006000	0x00007000	0x0000D000
virtual-address (end)	0x00002F65	0x00004E44	0x000057E4	0x00006009	0x0000CC58	0x0000D6D0
virtual-size (43198 bytes)	0x00001F65 (8037 bytes)	0x00001E44 (7748 bytes)	0x000007E4 (2020 bytes)	0x00000009 (9 bytes)	0x00005C58 (23640 bytes)	0x000006D0 (1744 bytes)
characteristics	0x60000020	0x40000040	0xC0000040	0xC0000040	0x40000040	0x42000040
write	-	-	x	x	-	-
execute	x	-	-	-	-	-
share	-	-	-	-	-	-
self-modifying	-	-	-	-	-	-
virtual	-	-	-	-	-	-

Виджу высокую энтропию по секциям .text, .rsrc, .reloc

library (10)	flag (0)	type	imports (232)	description
mfcl140.dll	-	Implicit	<u>173</u>	n/a
KERNEL32.dll	-	Implicit	<u>22</u>	Windows NT BASE API Client
USER32.dll	-	Implicit	<u>8</u>	Multi-User Windows USER API Client Library
COMCTL32.dll	-	Implicit	<u>1</u>	Common Controls Library
VCRUNTIME140.dll	-	Implicit	<u>4</u>	Microsoft C Runtime Library
api-ms-win-crt-h...	-	Implicit	<u>2</u>	Windows ApiSet Stub Library
api-ms-win-crt-r...	-	Implicit	<u>17</u>	Windows ApiSet Stub Library
api-ms-win-crt-...	-	Implicit	<u>1</u>	Windows ApiSet Stub Library
api-ms-win-crt-s...	-	Implicit	<u>2</u>	Windows ApiSet Stub Library
api-ms-win-crt-l...	-	Implicit	<u>2</u>	Windows ApiSet Stub Library

Импорты, намекающие на использование приемов анти-отладки (использованы в контексте, похожем на системный)

Import
InitializeCriticalSection
GetProcAddress
CreateEvent
GetSystemTimeAsFileTime
GetCurrentThreadId
GetCurrentProcessId

Import
QueryPerformanceCounter
GetStartupInfo
IsDebuggerPresent
IsProcessorFeaturePresent
TerminateProcess
GetCurrentProcess
OutputDebugString

На контекст использования этих строк надо будет обратить пристальное внимание - скорее всего, они относятся к основной логике, присутствует ступенчатая валидация ввода

ascii	27	0x00002878	-	InitializeConditionVariable
ascii	24	0x00002894	-	SleepConditionVariableCS
ascii	24	0x000028B0	-	WakeAllConditionVariable
ascii	3	0x000028CC	-	xQ@
ascii	3	0x000028D4	-	((I
ascii	38	0x0000296C	-	Local AppWizard-Generated Applications
ascii	3	0x00002994	-	t>@
ascii	9	0x00002AE8	-	kaspersky
ascii	5	0x00002AF8	-	Error
ascii	35	0x00002B00	-	Please, specify your name or email.
ascii	34	0x00002B24	-	Please, specify the serial number.
ascii	67	0x00002B48	-	The name or email and the serial should be less then %u chars long.
ascii	57	0x00002B8C	-	The name or email field should contains only ASCII chars.
ascii	50	0x00002BC8	-	The serial number should contains only hex digits.
ascii	39	0x00002BFC	-	The length of the serial should be odd.
ascii	8	0x00002C24	-	Correct!
ascii	26	0x00002C30	-	Good job! Congratulations!
ascii	19	0x00002C4C	-	Wrong! Try again...

Моё внимание привлекли и такие строки - но они уже относятся к ресурсам

ascii	15	0x0000442C	-	?.AVtype_info@@
ascii	16	0x00004444	-	?.AVCCmdTarget@@
ascii	13	0x00004460	-	?.AVCObject@@
ascii	13	0x00004478	-	?.AVCWinApp@@
ascii	16	0x00004490	-	?.AVCWinThread@@
ascii	16	0x000044AC	-	?.AVCloaderApp@@
ascii	13	0x000044C8	-	?.AVCDialog@@
ascii	16	0x000044E0	-	?.AVCloaderDlg@@
ascii	14	0x000044FC	-	?.AVCListBox@@
ascii	10	0x00004514	-	?.AVCWnd@@
ascii	22	0x00004B69	-	www
ascii	22	0x00004D79	-	www
ascii	3	0x00004DDB	-	www
ascii	4	0x00004DE9	-	www
ascii	12	0x00004F8E	-	www}
ascii	8	0x00004FD8	-	www
ascii	14	0x000051D1	-	www
ascii	14	0x000052B1	-	www
ascii	8	0x00005394	-	www}
ascii	4	0x000053C6	-	www
ascii	9	0x000054B8	-	www}x
ascii	9	0x000054F7	-	}www}
ascii	4	0x00005532	-	www
ascii	3	0x000055AC	-	wXk
ascii	3	0x000055B0	-	qN]
ascii	3	0x000055B4	-	tno
ascii	3	0x00005830	-	<sa
ascii	19	0x00005A03	-	_ _ _ _ 'a' _ _ _ _

Открываем IDA

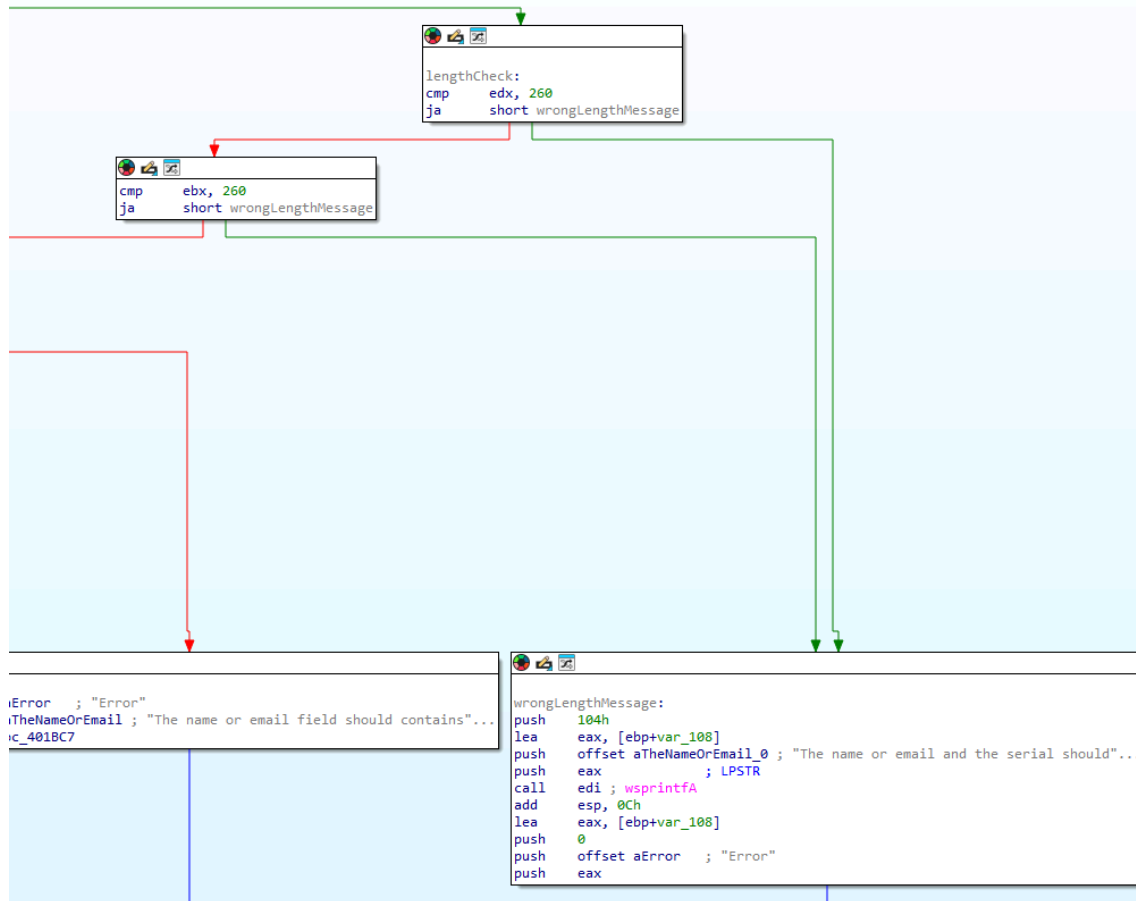
По кросс-референсу выходим на главную логику

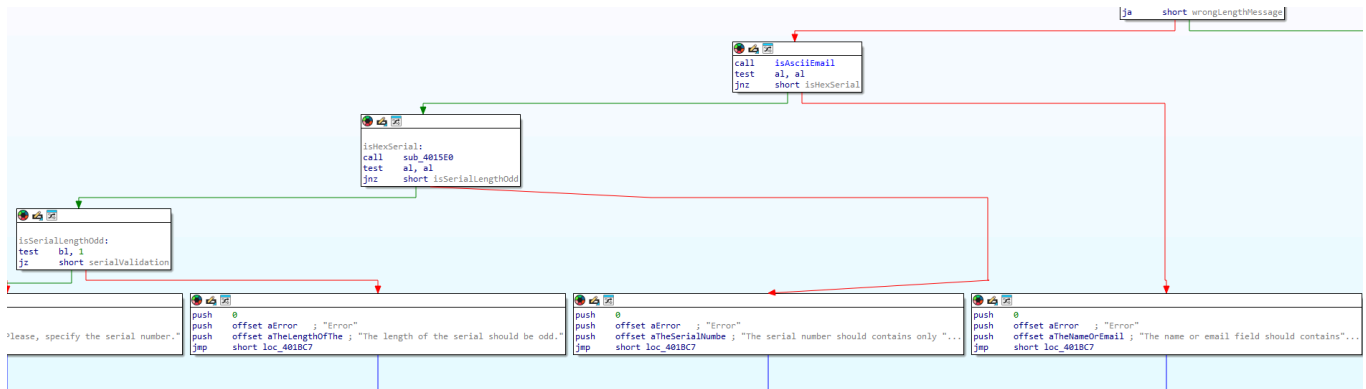
```

.rdata:00403700 aPleaseSpecifyY db "Please, specify your name or email.",0
.rdata:00403700 ; DATA XREF: sub_401A80+65fo
.rdata:00403724 aPleaseSpecifyT db "Please, specify the serial number.",0
.rdata:00403724 ; DATA XREF: sub_401A80+A1fo
.rdata:00403747 align 4
.rdata:00403748 ; const CHAR aTheNameOrEmail_0[]
.rdata:00403748 aTheNameOrEmail_0 db "The name or email and the serial should be less then %u chars lon'
.rdata:00403748 ; DATA XREF: sub_401A80+12Efo
.rdata:00403789 db 'g.',0
.rdata:0040378C aTheNameOrEmail db "The name or email field should contains only ASCII chars.",0
.rdata:0040378C ; DATA XREF: sub_401A80+CBfo
.rdata:004037C6 align 4
.rdata:004037C8 aTheSerialNumbe db "The serial number should contains only hex digits.",0
.rdata:004037C8 ; DATA XREF: sub_401A80+E2fo
.rdata:004037F8 align 4
.rdata:004037FC aTheLengthOfThe db "The length of the serial should be odd.",0
.rdata:004037FC ; DATA XREF: sub_401A80+F5fo
.rdata:00403824 aCorrect db "Correct!",0
.rdata:00403824 ; DATA XREF: sub_401A80+109fo
.rdata:0040382D align 10h
.rdata:00403830 aGoodJobCongrat db "Good job! Congratulations!",0
.rdata:00403830 ; DATA XREF: sub_401A80+10Efo
.rdata:00403848 align 4
.rdata:0040384C aWrongTryAgain db
.rdata:0040384C dd
.rdata:00403860 off_403860
.rdata:00403860 dd
.rdata:00403864 unk_403868
.rdata:00403868 db
.rdata:00403869 db
.rdata:0040386A db
.rdata:0040386B db
.rdata:0040386C db
.rdata:0040386D db
.rdata:0040386E db
.rdata:0040386F db
.rdata:00403870 db
.rdata:00403871 db
.rdata:00403872 db

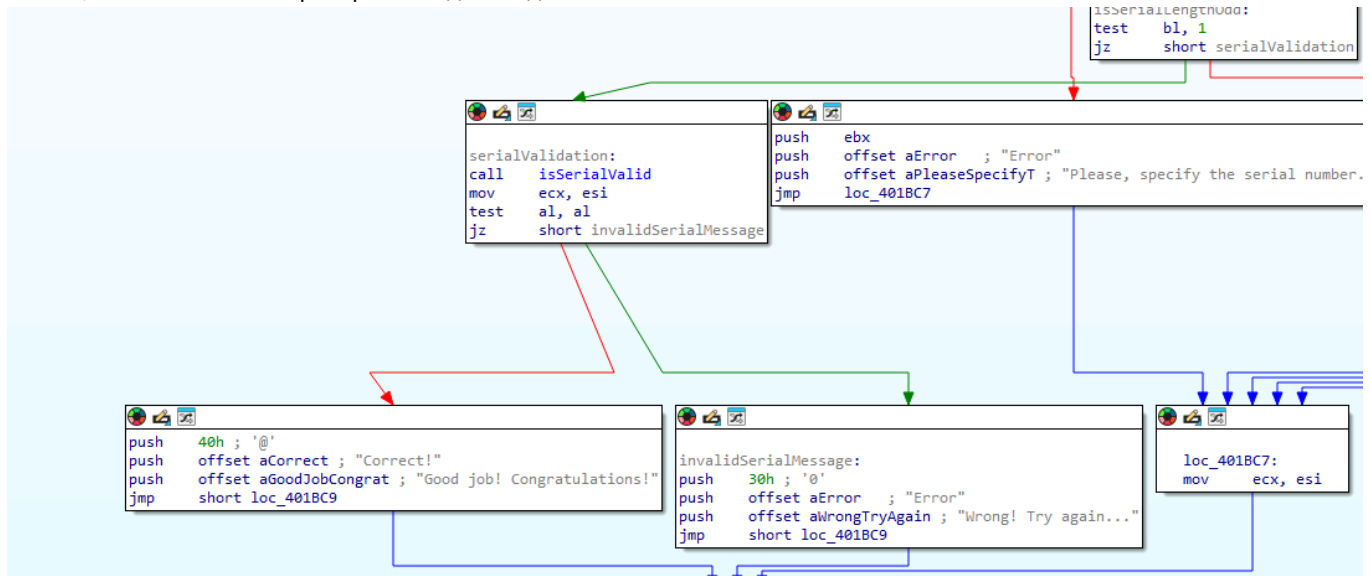
```

Типичные проверки, исходя из контекста:





Значит, основная логика проверки находится здесь:



Судя по всему, перед проверкой происходят ещё несколько преобразований\проверок введенного серийника -

```

char isSerialValid()
{
    HANDLE ProcessHeap; // eax
    int v1; // ecx
    int v2; // ecx
    unsigned int v4; // [esp+0h] [ebp-120h] BYREF
    const char *v5; // [esp+4h] [ebp-11Ch] BYREF
    BYTE v6[260]; // [esp+8h] [ebp-118h] BYREF
    char kaspersky[10]; // [esp+10Ch] [ebp-14h] BYREF
    int v8; // [esp+116h] [ebp-Ah]
    __int16 v9; // [esp+11Ah] [ebp-6h]

    if ( strlen(byte_405600) < 4 || strlen(&byte_4054F8) < 0x20 )
        return 0;
    ProcessHeap = GetProcessHeap();
    v5 = (const char *)HeapAlloc(ProcessHeap, HEAP_ZERO_MEMORY, 0x104u);
    strlen(&byte_4054F8);
    if ( (unsigned __int8)sub_401540(v6) )
    {
        v8 = 0;
        v9 = 0;
        v4 = 0;
        strcpy(kaspersky, "kaspersky");
        if ( !sub_401840(&v5, &v4, kaspersky, v1) && strlen(byte_405600) == strlen(v5) )
        {
            v2 = 0;
            if ( !v4 )
                return 1;
            while ( byte_405600[v2] == v5[v2] )
            {
                if ( ++v2 >= v4 )
                    return 1;
            }
        }
    }
    return 0;
}

```

Что я вижу:

```
if(strlen(byte_405600) < 4 || strlenmm(&byte_4054f8) < 32)
```

Надо понять, что в них хранится. Посмотрим на первое

Direction	Type	Address	Text
Up	o	isAsciiEmail	mov edx, offset byte_405600
Up	r	isAsciiEmail:loc_401650	mov cl, byte_405600[edx]
Up	o	isValidSerial+13	mov ecx, offset byte_405600
Up	o	isValidSerial+E0	mov ecx, offset byte_405600
Up	o	isValidSerial+11A	sub esi, offset byte_405600
Up	r	isValidSerial:loc_401A30	mov al, byte_405600[ecx]
Up	r	isValidSerial+126	cmp al, byte_405600[esi+ecx]
Up	o	sub_401A80+30	push offset byte_405600; LPSTR
Up	o	sub_401A80+49	mov edx, offset byte_405600

И первое, и второе устанавливаются в главной функции, обрабатывающей пользовательский ввод

```

1 int __thiscall sub_401A80(const char **this)
2 {
3     unsigned int n0x104; // kr00_4
4     unsigned int n0x104_1; // kr04_4
5     CHAR v5[260]; // [esp+8h] [ebp-108h] BYREF
6
7     CWnd::UpdateData((CWnd *)this, 1);
8     wsprintfA(byte_405600, "%s", *(this + 76));
9     wsprintfA(&byte_4054F8, "%s", *(this + 77));
10    n0x104 = strlen(byte_405600);
11    if ( !n0x104 )
12    {
13        return CWnd::MessageBoxA((CWnd *)this, "Please, specify your name or email.", "Error", 0);
14        n0x104_1 = strlen(&byte_4054F8);
15        if ( !n0x104_1 )
16        {
17            return CWnd::MessageBoxA((CWnd *)this, "Please, specify the serial number.", "Error", 0);
18        }
19        if ( n0x104 > 0x104 || n0x104_1 > 0x104 )
20        {
21            wsprintfA(v5, "The name or email and the serial should be less then %u chars long.", 260);
22            return CWnd::MessageBoxA((CWnd *)this, v5, "Error", 0);
23        }
24    }
25    else if ( (unsigned __int8)isAsciiEmail(&byte_4054F9) )
26    {
27        if ( (unsigned __int8)hexCheck() )
28        {
29            if ( (n0x104_1 & 1) != 0 )
30            {
31                return CWnd::MessageBoxA((CWnd *)this, "The length of the serial should be odd.", "Error", 0);
32            }
33            else if ( isValidSerial() )
34            {
35                return CWnd::MessageBoxA((CWnd *)this, "Good job! Congratulations!", "Correct!", 0x40u);
36            }
37            else
38            {
39                return CWnd::MessageBoxA((CWnd *)this, "Wrong! Try again...", "Error", 0x30u);
40            }
41        }
42        else
43        {
44            return CWnd::MessageBoxA((CWnd *)this, "The serial number should contains only hex digits.", "Error", 0);
45        }
46    }
47    else
48    {
49        return CWnd::MessageBoxA((CWnd *)this, "The name or email field should contains only ASCII chars.", "Error", 0);
50    }
51 }
52
53 n0x104 = strlen(byte_405600);
54 if ( !n0x104 )
55     return CWnd::MessageBoxA((CWnd *)this, "Please, specify your name or email.", "Error", 0);
56 n0x104_1 = strlen(&byte_4054F8);
57 if ( !n0x104_1 )

```

Значит, первое - имя\email, второе - серийник

Идём дальше смотреть в функцию

```
if(sub_401540(v6))
```

```
v6=(&serialKey + v4);
```

v6 - указатель на элемент строки serialKey + v4

Значит, можно переименовать их соответственно.

```

1 offset = 0;
2 if ( !a2 )
3     return 1;
4 while ( 1 )
5 {
6     symbol = (&serialKey + offset);
7     if ( (unsigned __int8)(symbol - '0') > 9u )
8     {
9         if ( (unsigned __int8)(symbol - 'a') > 5u )
10        {
11            if ( (unsigned __int8)(symbol - 'A') > 5u )
12                return 0;
13            v7 = symbol - '7';
14        }
15        else
16        {
17            v7 = symbol - 'W';
18        }
19    }
20    else
21    {
22        v7 = symbol - '0';
23    }
24    v8 = byte_4054F9[offset];
25    if ( (unsigned __int8)(v8 - '0') > 9u )
26        break;
27    v9 = v8 - '0';
28 LABEL_14:
29    offset += 2;
30    *a3++ = v9 + 16 * v7;
31    if ( offset >= a2 )
32        return 1;
33 }
34 if ( (unsigned __int8)(v8 - 'a') <= 5u )
35 {
36     v9 = v8 - 87;
37     goto LABEL_14;
38 }
39 if ( (unsigned __int8)(v8 - 'A') <= 5u )
40 {
41     v9 = v8 - 0x37;
42     goto LABEL_14;
43 }
44 return 0;
45 }

```

Пойдем по проверке в цикле while(1)

Похоже на вытаскивание hex-значений из строки. Почему я так думаю?

У нас 9 цифр, и тогда символ передается разницей ascii-представления цифр, если ascii-код символа не меньше 0x39 - 9, и так далее по строчным и прописным буквам английского алфавита.

Если у нас символ, например, 'a' - 0x61, тогда $0x61 - 0x57 = 0xA$

Если символ 'A', то $0x41 - 0x37 = 0xA$

Затем к оффсету прибавляется два, так пока пока он не станет больше или равен заданной длине - то есть разбирается по полубайту и комбинируется в два байта

И, переименовав переменные для удобного восприятия, получаем примерно такое:

```

char __fastcall isHexSerial(int a1, unsigned int offset, _BYTE *resultBuffer)
{
    unsigned int offset_1; // esi
    char symbol; // cl
    char firstHalf; // cl
    char anotherHex; // dl
    char secondHalf; // al

    offset_1 = 0;
    if ( !offset )
        return 1;
    while ( 1 )
    {
        symbol = (&serialKey + offset_1);
        if ( (unsigned __int8)(symbol - '0') > 9u )
        {
            if ( (unsigned __int8)(symbol - 'a') > 5u )
            {
                if ( (unsigned __int8)(symbol - 'A') > 5u )
                    return 0;
                firstHalf = symbol - '7';
            }
            else
            {
                firstHalf = symbol - 'W';
            }
        }
        else
        {
            firstHalf = symbol - '0';
        }
        anotherHex = byte_4054F9[offset_1];
        if ( (unsigned __int8)(anotherHex - '0') > 9u )
            break;
        secondHalf = anotherHex - '0';
    LABEL_14:
        offset_1 += 2;
        *resultBuffer++ = secondHalf + 16 * firstHalf;
        if ( offset_1 >= offset )
            return 1;
    }
    if ( (unsigned __int8)(anotherHex - 'a') <= 5u )
    {
        secondHalf = anotherHex - 87;
        goto LABEL_14;
    }
    if ( (unsigned __int8)(anotherHex - 'A') <= 5u )
    {
        secondHalf = anotherHex - 0x37;
        goto LABEL_14;
    }
    return 0;
}

```

resultBuffer - обработанная строка, состоящая из хекс-значений ключа

Двигаемся дальше к другому вызову функции

```

strcpy(kaspersky, "kaspersky");
if(! (sub_401840(&v5, &v4, kaspersky, v1)) && strlen(email) == strlen(v5))

```

Возможно, "kaspersky" - seed/initial key, если в sub_401840 я встречу что-то, похожее на криптографию

Используются также две переменные: v5, v4

v5 задается как


```
ProcessHeap=getProcessHeap();
v5=(const char*)HeapAlloc(ProcessHeap, HEAP_ZERO_MEMORY, 260)
```

По спецификации

```
DECLSPEC_ALLOCATOR LPVOID HeapAlloc
(
[in] HANDLE hHeap,
[in] DWORD dwFlags,
[in] SIZE_T dwBytes
);
```

Значит, в куче выделяется буфер размером 260 символов (так как символ char занимает байт в памяти) - 260 байт
И в v5 записывается указатель на его начало.

v4 перед вызовом принимает значение 0

```
v8 = 0;
v9 = 0;
v4 = 0;
strcpy(seed, "kaspersky");
if ( !((__DWORD (__stdcall *)(const char **, unsigned int *, char *, int))sub_401840)(&bufferPtr, &v4, seed, v1)
```

Идем в функцию, но как-то многовато тут аргументов - передавалось-то четыре.

```
int __fastcall sub_401840(int a1, int a2, const char **a3, unsigned int *a4, char *kaspersky, int a6)
{
```

Передавалось две ссылки на массивы символов, значит, **a3 - указатель на выделенный ранее буфер, причем судя по логике, буфер результирующий

Также выделяется ещё один буфер. В результате пока выглядит так

```
1 int __fastcall sub_401840(int a1, int a2, const char **resultBuffer, unsigned int *a4, char *kaspersky, int a6)
2 {
3     unsigned int v6; // edi
4     HANDLE ProcessHeap; // eax
5     const char *anotherBuffer; // esi
6     unsigned int v9; // edi
7     SIZE_T v11; // [esp-4h] [ebp-DCh]
8     unsigned int v12; // [esp+18h] [ebp-C0h]
9     _BYTE v13[180]; // [esp+20h] [ebp-B8h] BYREF
10
11     v6 = a2 & 4294967280;
12     v12 = a2 & 4294967280;
13     if ( !a1 )
14         return 8;
15     v11 = a2 & 4294967280;
16     ProcessHeap = GetProcessHeap();
17     anotherBuffer = (const char *)HeapAlloc(ProcessHeap, HEAP_ZERO_MEMORY, v11);
18     if ( !anotherBuffer )
19         return 8;
20     sub_401670(v13, kaspersky);
21     v9 = v6 >> 4;
22     *resultBuffer = anotherBuffer;
23     for ( *a4 = v12; v9; --v9 )
24     {
25         sub_401790(anotherBuffer);
26         anotherBuffer += 16;
27     }
28     return 0;
29 }
```

Заглянем в sub_401670

Взгляд цепляется за три блока кода

```

24 | do
25 | {
26 |     v6 = *(unsigned __int8 *)(n16 + kaspersky);
27 |     ++n16;
28 |     v5 = v6 | (v5 << 8);
29 |     if ( (n16 & 3) == 0 )
30 |     {
31 |         v19[n44_1++] = v5;
32 |         v5 = 0;
33 |     }
34 | }
35 | while ( n16 < 16 );
36 | dst_1[0] = 0xB7E15163;
37 | for ( i = 1; i < 44; ++i )
38 |     dst_1[i] = v19[i + 63] - 0x61C88647;
50 | if ( result )
51 | {
52 |     do
53 |     {
54 |         v12 = __ROR4__(v11 + dst_1[v10] + v12, 3);
55 |         dst_1[v10] = v12;
56 |         v11 = __ROR4__(v12 + v19[v9] + v11, v11 + v12);
57 |         v19[v9] = v11;
58 |         v10 = (v18 + 1) % 44;
59 |         v18 = v10;
60 |         result = (v17 + 1) / n44_1;
61 |         v9 = (v17 + 1) % n44_1;
62 |         v14 = v16-- == 1;
63 |         v17 = v9;
64 |     }
65 |     while ( !v14 );
66 | }
67 | memcpy(dst, dst_1, 176u);

```

Из последнего скриншота выходит какая-то криптографическая логика со сдвигами вправо

На втором скриншоте две константы, которые выглядят как магические числа - не как случайные.

Загуглим их



Amazon-themed campaigns of Lazarus in the Netherlands and Belgium

[welivesecurity.com](#) › 2022/09/30/amazon-themed-...

The constants **0xB7E15163** and **0x61C88647** (which is **-0x9E3779B9**; see Figure 6, Line 29 & 35) in the key expansion suggests that it's either the RC5 or RC6 algorithm. By checking the main decryption loop of the algorithm, one identifies that it's the more complex of the two...



c++ - RC5 Decrypt/Encrypt for Conquer game client - Stack Overflow

[stackoverflow.com](#) › questions/4955965/rc5-decrypt-...

Looks like `rotate_left` and `rotate_right` are the same, but they shouldn't be: unsigned long `rotate_left(unsigned long nData, unsigned long nCount)` { return (nData << (nCount & 0x1F) | nData >> 0x20 - (nCount & 0x1F))



EQUATION GROUP

[wikileaks.org](#) › ciav7p1/cms/files/Equation_group_...

One immediately notices the constants **0xB7E15163** and **0x61C88647**. Here's what a normal RC6 key setup code looks like ... Inside the Equation group malware, the encryption library uses a subtract operation with the constant **0x61C88647**.

PDF

Посмотреть

Получается, используется шифр семейства RC.

Расширенный ключ должен уже быть в памяти после выполнения

```
qmemcpy(dst, dst_1, 176u)
```

В ассемблерном листинге

```
.text:00B7176C loc_B7176C:                                ; CODE XREF: keyExpansion+95↑j
.text:00B7176C      mov     edi, [ebp+var_1DC]
.text:00B71772      lea     esi, [ebp+var_CC]
.text:00B71778      mov     ecx, 2Ch ; ','
.text:00B7177D      rep movsd
.text:00B7177F      mov     ecx, [ebp+var_4]
.text:00B71782      pop     edi
.text:00B71783      pop     esi
.text:00B71784      xor     ecx, ebp ; StackCookie
.text:00B71786      pop     ebx
.text:00B71787      call   @__security_check_cookie@4 ; __security_check_cookie(x)
.text:00B7178C      mov     esp, ebp
.text:00B7178E      pop     ebp
.text:00B7178F      retn
.text:00B7178F keyExpansion endp
```

`movsd` копирует из `esi` в `edi` - а там адреса `ebp+var_CC` и `ebp+var_1DC` соответственно.

Копирует 176 байт

Перейдем к дампу по адресу, после выполнения расширения ключа

Скриншот окна отладчика, отображающего код сборки и контекстное меню. В окне сборки видны инструкции, такие как:

```

00BE175D 83AD 28FEFFFF 01 sub dword ptr ss:[ebp-1D8],1
00BE1764 8995 2CFEFFFF mov dword ptr ss:[ebp-1D4],edx
00BE176A 75 A4 jne crackme.BE1710
00BE176C 88BD 24FEFFFF mov edi,dword ptr ss:[ebp-1DC]
00BE1772 80B5 34FEFFFF lea esi,dword ptr ss:[ebp-CC]
00BE1778 B9 2C000000 mov ecx,2C
00BE177F F3A5 rep movsd
00BE1782 88D FC mov ecx,dword ptr ss:[ebp-4]
00BE1783 5F pop edi
00BE1784 5E pop esi
00BE1785 33CD xor ecx,ebp
00BE1786 58 pop ebx
00BE1787 E8 8A080000 call crackme.BE2016
00BE178C 88E5 mov esp,ebp
00BE178E 5D pop ebp
00BE178F C3 ret
00BE1790 55 push ebp
00BE1791 88EC mov ebp,esp
00BE1793 83EC 10 sub esp,10
00BE1796 8842 08 mov eax,dword ptr ds:[edx+8]
00BE1799 53 push ebx
00BE179A 885A 04 mov ebx,dword ptr ds:[edx+4]
00BE179D 56 push esi
00BE179E 88F1 mov esi,ecx
00BE17A0 C745 F4 13000000 mov dword ptr ss:[ebp-C],13
00BE17A7 884A 0C mov ecx,dword ptr ds:[edx+C]
00BE17AA 57 push edi
00BE17AB 883A mov edi,dword ptr ds:[edx]
00BE17AD 28BE A8000000 sub edi,dword ptr ds:[esi+A]
00BE17B3 8096 A0000000 lea edx,dword ptr ds:[esi+A]
00BE17B9 28B6 AC000000 sub eax,dword ptr ds:[esi+A]
00BE17BF 8975 F0 mov dword ptr ss:[ebp-10],esi
00BE17C2 8955 FC mov dword ptr ss:[ebp-4],edi
00BE17C5 EB 03 jmp crackme.BE17CA
00BE17C7 884D F8 mov ecx,dword ptr ss:[ebp-8]
00BE17CA 8BD0 mov edx,eax
00BE17CC 8BC3 mov eax,ebx

```

Контекстное меню, появившееся над кодом, содержит следующие пункты:

- Двоичные операции
- Копировать
- Точка останова
- Перейти к дампу
- Перейти к дизассемблированному коду
- Перейти к карте памяти
- График
- Справка по мнемонике
- Показать краткое описание мнемоники
- Режим подсветки
- Редактирование столбцов...
- Метка
- Комментарий
- Поставить/Убрать закладку
- Trace coverage
- Анализ
- Ассемблировать
- Исправления
- Установить EIP здесь
- Создать новый поток здесь
- Перейти
- Поиск в
- Поиск ссылок на

В нижней части экрана видна панель дампа, содержащая шестнадцатеричные и ASCII данные:

Адрес	Шестнадцатеричное	ASCII
0099EA04	F7 20 9D 4E 6E 34 F4 08 35 F3 99 0B 88 2A 4B 4E	÷ .Nn4ô.5ó...*KN
0099EA14	FB 2D A1 F6 F6 DE 8B BF 25 F5 DD 83 BF 41 B7 86	û-ïöð.¿%öŸ.¿A·.
0099EA24	52 8B 60 5A DC F9 EF 7B 26 29 0E 34 0A 7E 4F 86	R. `ZÜüi{&).4.~O.
0099EA34	2B 24 A0 5E AC 35 A4 D8 47 3A F1 DC E2 5D 52 B7	+\$ ^~5я0G:ñÜà]R·
0099EA44	E1 16 5C 7B 82 66 74 9E 91 EF C8 21 DA A4 76 50	á.\{.ft..îË!ÚπvP
0099EA54	2E 9B D3 34 D2 34 66 80 6A D0 40 1D 15 B1 22 EE	. .ô404f.jð@. .±"i
0099EA64	C2 BC 74 3B 2B 26 BC B3 97 84 3F 9B 35 3C 19 C7	Â%t;+&%³..?.5< .Ç
0099EA74	72 01 18 5D DE B6 50 57 03 6B 20 07 33 47 37 3A	r..]p¶PW.k .3G7:
0099EA84	E2 56 16 F6 E5 B8 59 E7 29 C3 85 FE 06 10 8D 1F	âV.ôâ,Yç)Ã.p....
0099EA94	31 F6 39 70 C1 CC 8C 06 ED D4 F8 C9 72 6A C3 F5	1ô9pÃÎ..iôøËrjÃö
0099EAA4	56 13 3E 82 4D E5 24 96 4E 6C DA A6 84 85 CB 57	V.>.Mã\$.NLÚ!..ËW

С другой стороны,

```
!cipherlogic((int)hexSerial, offset>>1, &bufferPtr, &v5, seed, v2) && strlen(email)==strlen(bufferPtr)
```

Может, наш серийник - это зашифрованный email с ключом kaspersky по алгоритму RC6?

Посмотрим снова на основную криптологику

```

v3 = a2[1];
n19 = 19;
v5 = a2[3];
v6 = *a2 - dst[42];
v7 = a2[2] - dst[43];
dst_1 = dst;
v19 = dst + 40;
while ( 1 )
{
    v8 = v7;
    v9 = v3;
    v3 = v6;
    v18 = v8;
    v10 = v5;
    v11 = __ROR4__(v8 * (2 * v8 + 1), 5);
    v12 = __ROR4__(v3 * (2 * v3 + 1), 5);
    v13 = v19;
    v7 = v11 ^ __ROL4__(v9 - v19[1], v12);
    v19 -= 2;
    v6 = v12 ^ __ROL4__(v10 - *v13, v11);
    if ( --n19 < 0 )
        break;
    v5 = v18;
}
anotherBuffer[2] = v7;
v14 = v3 - *dst_1;
result = v18 - dst_1[1];
*anotherBuffer = v6;
anotherBuffer[1] = v14;
anotherBuffer[3] = result;
return result;
}

```

```

B = B + S[0]
D = D + S[1]
for i = 1 to r do
{
    t = (B(2B + 1)) <<< lg w
    u = (D(2D + 1)) <<< lg w
    A = ((A ⊕ t) <<< u) + S[2i]
    C = ((C ⊕ u) <<< t) + S[2i + 1]
    (A, B, C, D) = (B, C, D, A)
}
A = A + S[2r + 2]
C = C + S[2r + 3]

```

Процедура расшифровки:

Вход:

- зашифрованный текст, сохранённый в A, B, C и D
- r количество раундов
- w-разрядные ключи для каждого раунда S[0, ..., 2r + 3]

Выход:

- исходный текст сохраняется в A, B, C и D

```

C = C - S[2r + 3]
A = A - S[2r + 2]

for i = r downto 1 do
{
    (A, B, C, D) = (D, A, B, C)
    u = (D(2D + 1)) <<< lg w
    t = (B(2B + 1)) <<< lg w
    C = ((C - S[2i + 1]) >>> t) ⊕ u
    A = ((A - S[2i]) >>> u) ⊕ t
}
D = D - S[1]
B = B - S[0]

```

В дизасме вычитания - значит, расшифровка

Резюмируем

Логика работы программы

1. Программа принимает на вход имейл и серийник
2. Идет череда проверок:
 - Имейл введен
 - Серийник введен

- Длина имейла не больше размера буфера
- Длина серийника не больше размера буфера
- Имейл должен состоять из печатных символов ASCII (не расширенная таблица)
- Серийник должен содержать символы алфавита хекс-кодов (0-9, A-F, a-f)
- Длина серийника должна быть нечетной (тут сомнения, 32 символа пропускает)

3. После прохождения проверок вызывается функция валидации серийника:

- Выделяется буфер в куче
- Серийник переводится в хекс-строку
- Копируется ключ `kaspersky`
- Происходит расширение ключа
- Происходит криптографическое преобразование серийника

Кроме того, серийник и имейл должны быть одной длины, чтобы попасть в цикл посимвольной проверки
Проверяются первые 0x10 - 16 символов.

Про криптографическое преобразование

В расширении ключа фигурируют две константы:

0xB7E15163

0x61C88647

Они используются в алгоритмах семейства RC

Происходит деление входной строки на блоки, 20 раундов шифрования.

Алгоритм не поточный - блочный, симметричный

Или RC5, или RC6. Константы расширения ключа указывают скорее на RC6

```

v6 = *a2 - a1[0x2A];
v7 = a2[2] - a1[43];
v16 = a1;
v19 = a1 + 40;
while ( 1 )
{
    v8 = v7;
    v9 = v3;
    v3 = v6;
    v18 = v8;
    v10 = v5;

    v11 = __ROR4__(v8 * (2 * v8 + 1), 5);
    v12 = __ROR4__(v3 * (2 * v3 + 1), 5);
    v13 = v19;
    v7 = v11 ^ __ROL4__(v9 - v19[1], v12);
    v19 -= 2;
    v6 = v12 ^ __ROL4__(v10 - *v13, v11);
    if ( --v17 < 0 )
        break;
    v5 = v18;
}
a3[2] = v7;
v14 = v3 - *v16;
result = v18 - v16[1];

```

Выделенные строки указывают на то, что:

- Ключи выбираются с конца массива
 - Происходит побитовый сдвиг влево
 - Большое количество вычитаний
- По базовой имплементации алгоритма это процесс дешифрования

Для поиска валидного серийника стоит использовать следующее свойство блочных шифров

```

Encrypt(plain,key) = cipher
Decrypt(cipher,key) = plain
OR
Decrypt(text,key)=plain
Encrypt(plain,key)=text

```

То есть если на вход функции дешифрования подать текст, а выход этой функции подать на вход функции шифрования, то мы должны получить текст, поданный на вход дешифрования.

Значит, надо инвертировать алгоритм дешифрования, чтобы он шифровал.

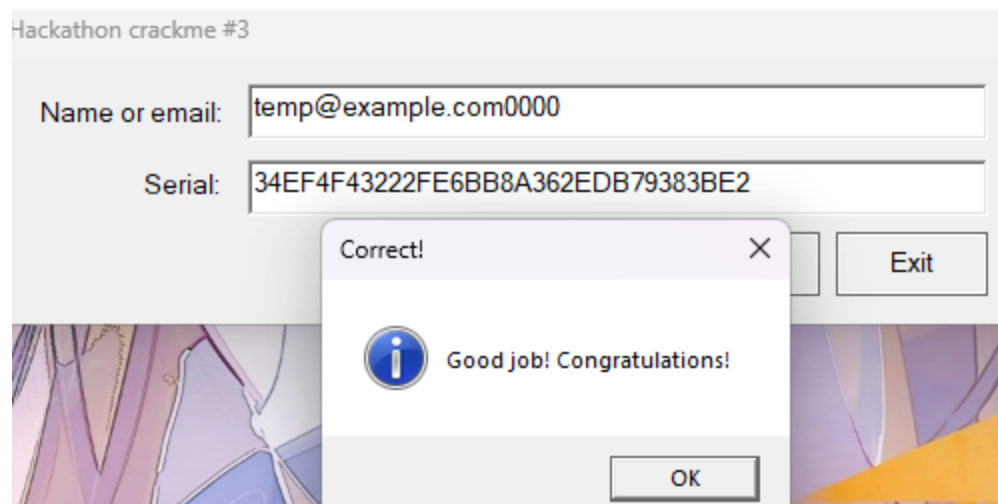
Удачный вариант решения

email: temp@example.com0000
Serial: 34EF4F43222FE6BB8A362EDB79383BE2

Решение

Я переписал процедуру дешифрования, и она совпала с классической RC6 Decryption
Написал для неё RC6 Encryption

```
int __fastcall Encryption(DWORD* keys, DWORD* plain, int* out)
{
    int A = *plain;
    int B = plain[1] + keys[0];
    int C = plain[2];
    int D = plain[3] + keys[1];
    for (int i = 1; i <= 20; i++)
    {
        int t = _rotr((unsigned int)(B * (2 * B + 1)), 5);
        int u = _rotr((unsigned int)(D * (2 * D + 1)), 5);
        A = _rotr((A ^ t), u) + keys[2 * i];
        C = _rotr((C ^ u), t) + keys[2 * i + 1];
        int tempA = A;
        int tempB = B;
        int tempC = C;
        int tempD = D;
        A = tempB;
        B = tempC;
        C = tempD;
        D = tempA;
    }
    A = A + keys[42];
    C = C + keys[43];
    out[0] = A;
    out[1] = B;
    out[2] = C;
    out[3] = D;
    return 1;
}
```



Неудачный вариант решения

Определенно позже стоит довести этот вариант решения до конца, но сразу он не сработал, так как я просто потерялся в порядке слов в отладчике

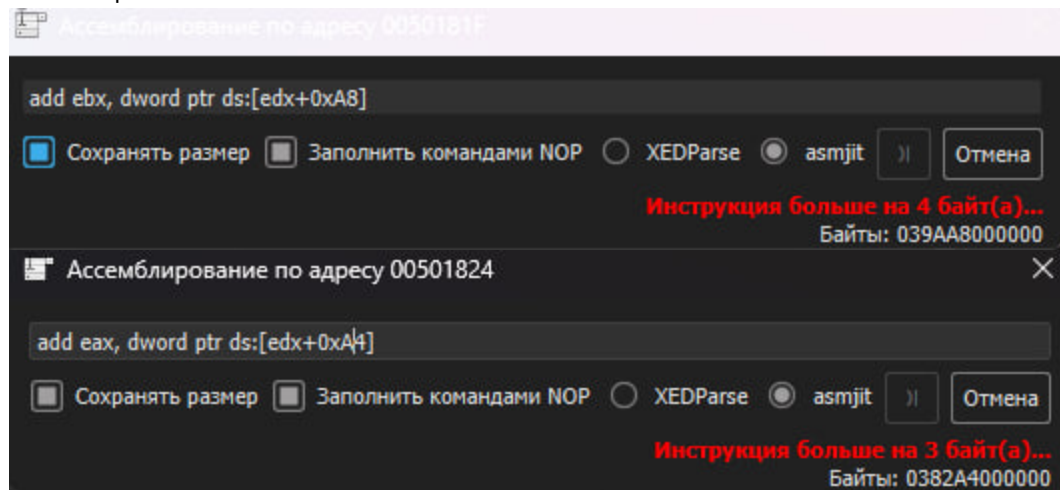
Патч

Идея - пропатчить логику дешифрования, чтобы вместо дешифрования происходило шифрования. Тогда можно будет перед вызовом этой функции переписать входной буфер на хекс-коды нужного нам имейла и посмотреть шифротекст на выходе.

Патчноут

```
mov [ebp-0xC], 0x13 -> mov [ebp-0xC], 0x0
sub edi,[esi+0xA8] -> add edi,[esi]
lea edx,[esi+0xA0] -> lea edx,[esi+0x8]
sub eax,[esi+0xAC] -> add eax,[esi+0x4]
ror edx, 5 -> rol edx, 5
ror esi, 5 -> rol esi, 5
rol eax, cl -> ror eax, cl
sub eax,[ecx+0x4] -> add eax,[ecx+0x4]
sub [ebp-0x4], 0x8 -> add [ebp-0x4], 0x8
sub edi,[ecx] -> add edi,[ecx]
ror edi, cl -> rol edi, cl
dec ecx -> inc ecx
jns 5017c7 -> jl 5017c7
sub ebx,[edx] -> add ebx,[edx+0xA8]
sub eax,[edx+0x4] -> add eax,[edx+0xAC]
```

Но есть проблема:



Может, стоит тогда найти какое нибудь место в программе и сделать там code save - записать эти инструкции сюда, а переход и возврат сделать через jmp?

Патч:

00841790	55	push ebp	
00841791	8BEC	mov ebp,esp	
00841793	83EC 10	sub esp,10	
00841796	8B42 08	mov eax,dword ptr ds:[edx+8]	
00841799	53	push ebx	
0084179A	8B5A 04	mov ebx,dword ptr ds:[edx+4]	
0084179D	56	push esi	esi:"w1w"
0084179E	8BF1	mov esi,ecx	esi:"w1w", ecx:"B "
008417A0	C745 F4 00000000	mov dword ptr ss:[ebp-C],0	[ebp-0C]:wcstombs+70
008417A7	8B4A 0C	mov ecx,dword ptr ds:[edx+C]	ecx:"B "
008417AA	57	push edi	
008417AB	8B3A	mov edi,dword ptr ds:[edx]	
008417AD	2B3E	sub edi,dword ptr ds:[esi]	esi:"w1w"
008417AF	90	nop	
008417B0	90	nop	
008417B1	90	nop	
008417B2	90	nop	
008417B3	8D56 08	lea edx,dword ptr ds:[esi+8]	
008417B6	90	nop	
008417B7	90	nop	
008417B8	90	nop	
008417B9	0346 04	add eax,dword ptr ds:[esi+4]	
008417BC	90	nop	
008417BD	90	nop	
008417BE	90	nop	
008417BF	8975 F0	mov dword ptr ss:[ebp-10],esi	[ebp-10]:&"яяяяpE#w"
008417C2	8955 FC	mov dword ptr ss:[ebp-4],edx	
008417C5	EB 03	jmp crackme.8417CA	
008417C7	8B4D F8	mov ecx,dword ptr ss:[ebp-8]	
008417CA	8BD0	mov edx,eax	
008417CC	8BC3	mov eax,ebx	
008417CE	8BDF	mov ebx,edi	
008417D0	8955 F8	mov dword ptr ss:[ebp-8],edx	
008417D3	8BF9	mov edi,ecx	ecx:"B "
008417D5	8B4D FC	mov ecx,dword ptr ss:[ebp-4]	
008417D8	8D1455 01000000	lea edx,dword ptr ds:[edx*2+1]	
008417DF	0FAF55 F8	imul edx,dword ptr ss:[ebp-8]	
008417E3	8D345D 01000000	lea esi,dword ptr ds:[ebx*2+1]	esi:"w1w"
008417EA	0341 04	add eax,dword ptr ds:[ecx+4]	ecx+04:"ë""
008417ED	0FAFF3	imul esi,ebx	esi:"w1w"
008417F0	C1C2 05	rol edx,5	
008417F3	C1C6 05	rol esi,5	esi:"w1w"
008417F6	8BCE	mov ecx,esi	ecx:"B ", esi:"w1w"
008417F8	D3C8	ror eax,c1	
008417FA	8B4D FC	mov ecx,dword ptr ss:[ebp-4]	
008417FD	33C2	xor eax,edx	
008417FF	8345 FC 08	add dword ptr ss:[ebp-4],8	
00841803	0339	add edi,dword ptr ds:[ecx]	ecx:"B "
00841805	8BCA	mov ecx,edx	ecx:"B "
00841807	D3CF	ror edi,c1	
00841809	8B4D F4	mov ecx,dword ptr ss:[ebp-C]	[ebp-0C]:wcstombs+70
0084180C	33FE	xor edi,esi	esi:"w1w"
0084180E	41	inc ecx	ecx:"B "
0084180F	894D F4	mov dword ptr ss:[ebp-C],ecx	[ebp-0C]:wcstombs+70
00841812	85C9	test ecx,ecx	ecx:"B "
00841814	7C B1	j1 crackme.8417C7	
00841816	8B4D 08	mov ecx,dword ptr ss:[ebp+8]	
00841819	8B55 F0	mov edx,dword ptr ss:[ebp-10]	[ebp-10]:&"яяяяpE#w"
0084181C	8941 08	mov dword ptr ds:[ecx+8],eax	ecx+08:NtCallEnclave+4
0084181F	EB 17	jmp crackme.841838	
00841821	8B45 F8	mov eax,dword ptr ss:[ebp-8]	
00841824	E9 78F9FFFF	jmp crackme.8411A1	
00841829	5F	pop edi	
0084182A	5E	pop esi	esi:"w1w"
0084182B	8959 04	mov dword ptr ds:[ecx+4],ebx	ecx+04:"ë""
0084182E	8941 0C	mov dword ptr ds:[ecx+C],eax	ecx+0C:NtCallEnclave+8
00841831	5B	pop ebx	
00841832	8BE5	mov esp,ebp	
00841834	5D	pop ebp	
00841835	C2 0400	ret 4	

Code-caves:

00841838	039A A8000000	add ebx,dword ptr ds:[edx+A8]
0084183E	EB E1	jmp crackme.841821

008411A1	0382 AC000000	add eax,dword ptr ds:[edx+AC]
008411A7	8939	mov dword ptr ds:[ecx],edi
008411A9	E9 78060000	jmp crackme.841829

Дизасм

Было

```

20 v3 = a2[1];
21 v17 = 0x13;
22 v5 = a2[3];
23 v6 = *a2 - a1[0x2A];
24 v7 = a2[2] - a1[43];
25 v16 = a1;
26 v19 = a1 + 40;
27 while ( 1 )
28 {
29     v8 = v7;
30     v9 = v3;
31     v3 = v6;
32     v18 = v8;
33     v10 = v5;
34     v11 = __ROR4__(v8 * (2 * v8 + 1), 5);
35     v12 = __ROR4__(v3 * (2 * v3 + 1), 5);
36     v13 = v19;
37     v7 = v11 ^ __ROL4__(v9 - v19[1], v12);
38     v19 -= 2;
39     v6 = v12 ^ __ROL4__(v10 - *v13, v11);
40     if ( --v17 < 0 )
41         break;
42     v5 = v18;
43 }
44 a3[2] = v7;
45 v14 = v3 - *v16;
46 result = v18 - v16[1];
47 *a3 = v6;
48 a3[1] = v14;
49 a3[3] = result;
50 return result;
51 }

```

Стало

```

20 v4 = a2[1];
21 v17 = 0;
22 v6 = a2[3];
23 v7 = *a2 - *a1;
24 v8 = a1[1] + a2[2];
25 v16 = a1;
26 v19 = a1 + 2;
27 while ( 1 )
28 {
29     v9 = v8;
30     v10 = v4;
31     v4 = v7;
32     v18 = v9;
33     v11 = v6;
34     v12 = __ROL4__(v9 * (2 * v9 + 1), 5);
35     v13 = __ROL4__(v4 * (2 * v4 + 1), 5);
36     v14 = v19;
37     v8 = v12 ^ __ROR4__(v19[1] + v10, v13);
38     v19 += 2;
39     v7 = v13 ^ __ROR4__(*v14 + v11, v12);
40     if ( ++v17 >= 0 )
41         break;
42     v6 = v18;
43 }
44 a3[2] = v8;
45 v15 = v16[42] + v4;
46 result = v16[43] + v18;
47 *a3 = v7;
48 a3[1] = v15;
49 a3[3] = result;
50 return result;
51 }

```

Произошла ещё замена счетчика обратно на 0x13, и inc обратно на dec - чтобы раунды проходили в полном объеме. Стратегия кажется жизнеспособной, но я определённо что-то упускаю.

Дополнительная идея: Angr

Можно попробовать подобрать пару имейл-серийник или только серийник для имейла с помощью символьного исполнения. Если дать на вход адрес буфера для серийника, задать имейл, а потом задать адрес инструкции в ветке успешной валидации ключа.